

Lab 10 Spring WebFlux & WebClient

1. Reactive Programming vs Blocking

Blocking Programming เป็นการทำงานที่ thread ต้องหยุดจนกว่างานปัจจุบันจะเสร็จ เช่น การรอข้อมูลจากฐานข้อมูลหรือ API ภายนอก จึงอาจใช้ thread จำนวนมากเมื่อมี request เข้ามาพร้อมกัน

Reactive Programming เป็นการทำงานแบบ asynchronous และ non-blocking โดยคืนข้อมูลในรูปแบบ Publisher ก่อน แล้วส่งค่าต่อเมื่อข้อมูลพร้อม ทำให้ thread ไม่ต้องยืนรอและสามารถนำไปทำงานอื่นได้ ระหว่างทางสามารถต่อ operator เช่น map(), filter() หรือ switchIfEmpty() ได้โดยไม่เรียก .block() จึงเหมาะกับระบบที่มีงาน I/O และต้องรองรับ request พร้อมกันจำนวนมาก

2. Mono vs Flux

Mono<T> ใช้เมื่อผลลัพธ์มีได้ 0 หรือ 1 ค่า เช่น การค้นหาสินค้าตาม id ด้วย findById(), การบันทึกสินค้าด้วย save() หรือการคืนราคาลงส่วนลด เพราะแต่ละคำขอควรได้ผลลัพธ์เพียงหนึ่งรายการหรือไม่พบข้อมูลเลย

Flux<T> ใช้เมื่อผลลัพธ์มีได้ 0 ถึงหลายค่า เช่น findAll() และ findByCategory() เนื่องจากคำขอเหล่านี้อาจส่งคืนค่าหลายรายการออกมาเป็น stream การเลือก Mono หรือ Flux จึงขึ้นอยู่กับจำนวนค่าที่ method นั้นควรคืน ไม่ได้ขึ้นอยู่กับความเร็วของ method

3. WebClient และ Method Chain

WebClient เป็น HTTP client ของ Spring WebFlux ที่ใช้เรียก REST API แบบ reactive และ non-blocking โดย .get() ใช้กำหนดว่า request เป็น HTTP GET, .uri() ใช้กำหนด path ของ endpoint และแทนค่าตัวแปร เช่น /products/{id}, .retrieve() ใช้เริ่มรับและประมวลผล HTTP response จาก server จากนั้น .bodyToMono(Product.class) ใช้แปลง response body ที่มีหนึ่งค่าเป็น Mono<Product> ส่วน .bodyToFlux(Product.class) ใช้แปลง response ที่มีหลายรายการเป็น Flux<Product> ตัวอย่าง chain คือ

```
client.get().uri("/products/{id}", id).retrieve().bodyToMono(Product.class)
```

ซึ่งยังคงเป็น reactive chain ตลอดทาง

4. Code ของ Repository, Service และ Controller

4.1 ProductRepository

```

J ProductRepository.java X GET-products.png X GET-products-l.png X GET-products-l-price.png X GET-products-category-Electronics.png X ProductService.java X POST-products.p
src > main > java > com > example > lab10 > repository > J ProductRepository.java > ProductRepository
22 * - Flux.fromIterable(list) คั้นหลายค่าจาก collection
23 */
24 public class ProductRepository {
25
26     // — In-memory storage —
27     private final Map<String, Product> store = new ConcurrentHashMap<>();
28
29     // — Constructor: ใส่ข้อมูลตัวอย่าง —
30     public ProductRepository() {
31         store.put("1", new Product(id: "1", name: "iPhone 15 Pro (673380039-7 SEC 2)",
32             category: "Electronics", brand: "Apple", stock: 50, price: 39900.0, discountType: "MEMBER"));
33         store.put("2", new Product(id: "2", name: "MacBook Air M3",
34             category: "Electronics", brand: "Apple", stock: 20, price: 49900.0, discountType: "NONE"));
35         store.put("3", new Product(id: "3", name: "Samsung Galaxy S24",
36             category: "Electronics", brand: "Samsung", stock: 30, price: 29900.0, discountType: "SEASONAL"));
37     }
38
39     // — 1. หา Product 1 รายการ —
40     /**
41      * TODO: คั้น Mono<Product> จาก store โดยใช้ id
42      * ถ้าไม่พบให้คืน Mono.empty()
43      *
44      * Hint: store.get(id) คั้น Product หรือ null
45      * ถ้า null ให้คืน Mono.empty()
46      * ถ้ามีค่าให้คืน Mono.just(product)
47      */
48     public Mono<Product> findById(String id) {
49         Product product = store.get(id);
50
51         if (product == null) {
52             return Mono.empty();
53         }
54
55         return Mono.just(product);
56     }

```

```

80
81     // — 4. ลบ Product —
82     /**
83      * TODO: ลบ product จาก store แล้วคืน Mono<Void>
84      *
85      * Hint: store.remove(id)
86      * แล้วคืน Mono.empty() คั้นค่า (Mono<Void>)
87      */
88     public Mono<Void> deleteById(String id) {
89         store.remove(id);
90         return Mono.empty();
91     }
92
93     // — 5. กรองตาม category —
94     /**
95      * TODO: คั้น Flux<Product> ที่ category ตรงกัน
96      *
97      * Hint: findAll()
98      * .filter(p -> p.getCategory().equalsIgnoreCase(category))
99      */
100     public Flux<Product> findByCategory(String category) {
101         return findAll()
102             .filter(p -> p.getCategory().equalsIgnoreCase(category));
103     }
104 }
105

```

ProductRepository ทำหน้าที่จัดเก็บและเข้าถึงข้อมูลสินค้าใน ConcurrentHashMap โดย findById() คั้น Mono<Product>, findAll() คั้น Flux<Product>, save() บันทึกและคืนสินค้าที่บันทึก, deleteById() คั้น Mono<Void> และ findByCategory() ใช้ filter() เพื่อส่งต่อเฉพาะสินค้าที่มี category ตรงกับเงื่อนไข

4.2 ProductService

```

J ProductRepository.java X GET-products.png X GET-products-1.png X GET-products-1-price.png X GET-products-category-Electronics.png X ProductService.java X POST-products.p
src > main > java > com > example > lab10 > service > J ProductService.java > Language Support for Java(TM) by Red Hat > ProductService > save(Product)
19 * .map(p -> ...) แปลงค่า
20 * .flatMap(p -> ...) async transform
21 * .defaultIfEmpty(...) fallback ถ้าว่าง
22 * .switchIfEmpty(Mono...) fallback Mono ถ้าว่าง
23 */
24 @Service
25 public class ProductService {
26
27     // — Constructor Injection (DIP – SOLID) —————
28     private final ProductRepository repository;
29
30     public ProductService(ProductRepository repository) {
31         this.repository = repository;
32     }
33
34     // — 1. ดึง Product 1 รายการ —————
35     /**
36      * TODO: เรียก repository.findById(id) แล้วคืนผล
37      * ถ้าไม่พบให้ throw RuntimeException("Product not found: " + id)
38      *
39      * Hint: repository.findById(id)
40      * .switchIfEmpty(Mono.error(new RuntimeException(...)))
41      */
42     public Mono<Product> getById(String id) {
43         return repository.findById(id)
44             .switchIfEmpty(
45                 Mono.error(new RuntimeException("Product not found: " + id))
46             );
47     }
48
49     // — 2. ดึง Product ทั้งหมด —————
50     /**

```

```

82     * TODO: เรียก repository.findByCategory(category) แล้วคืนผล
83     */
84     public Flux<Product> getByCategory(String category) {
85         return repository.findByCategory(category);
86     }
87
88     // — 6. คำนวณราคาหลังส่วนลด —————
89     /**
90      * TODO: หา Product จาก id แล้วคืน discountedPrice
91      *
92      * Hint: getById(id)
93      * .map(p -> p.getDiscountedPrice())
94      */
95     public Mono<Double> getDiscountedPrice(String id) {
96         return getById(id)
97             .map(p -> p.getDiscountedPrice());
98     }
99 }
100

```

ProductService เป็นชั้น business logic ระหว่าง Controller และ Repository โดย getById() ใช้ switchIfEmpty() จัดการกรณีไม่พบสินค้า, save() สร้าง UUID เมื่อไม่มี id, getAll(), delete() และ getByCategory() ส่งต่อ reactive publisher จาก Repository และ getDiscountedPrice() ใช้ map() แปลง Mono<Product> เป็น Mono<Double>

4.3 ProductController

```

src > main > java > com > example > lab10 > controller > J ProductController.java > Language Support for Java(TM) by Red Hat > ProductController > save(Product)
17 * GET /products → Flux<Product> (สิ่งทั้งหมด)
18 * GET /products/{id} → Mono<Product> (ตัวอย่างทำแล้ว)
19 * POST /products → Mono<Product> (บันทึก)
20 * DELETE /products/{id} → Mono<Void> (ลบ)
21 * GET /products/category/{cat} → Flux<Product> (กรอง)
22 * GET /products/{id}/price → Mono<Double> (ราคาหลังลด)
23 */
24 @RestController
25 @RequestMapping("/products")
26 public class ProductController {
27
28     // — Constructor Injection (DIP - SOLID) —
29     private final ProductService service;
30
31     public ProductController(ProductService service) {
32         this.service = service;
33     }
34
35     //
36     // ตัวอย่างที่ทำเสร็จแล้ว - ศึกษาแล้วทำ endpoint ที่เหลือ
37     //
38
39     /**
40     * GET /products/{id}
41     * คืน Mono<Product> - ค้นหา Product 1 รายการ
42     *
43     * ทดสอบ: GET http://localhost:8080/products/1
44     */
45     @GetMapping("/{id}")
46     public Mono<Product> getById(@PathVariable String id) {
47         return service.getById(id);
48     }
49
50     //
51     // TODO: เติม method body ดังต่อไปนี้

```

```

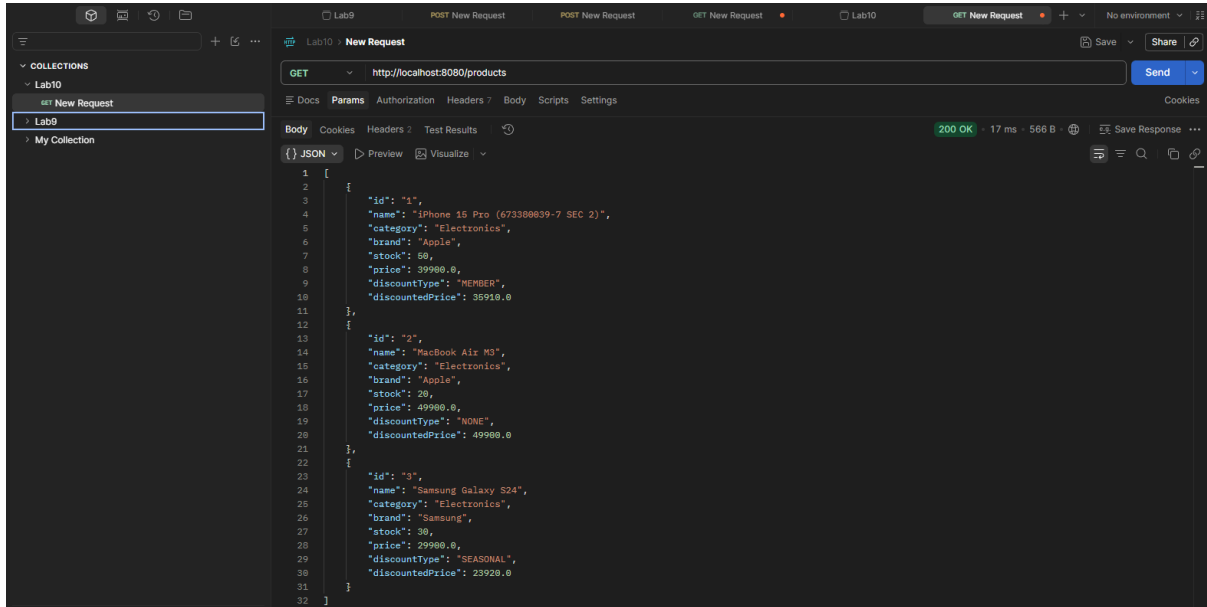
98 @GetMapping("/category/{category}")
99 public Flux<Product> getByCategory(@PathVariable String category) {
100     return service.getByCategory(category);
101 }
102
103 /**
104 * GET /products/{id}/price
105 * TODO: คืน Mono<Double> ราคาหลังส่วนลด
106 *
107 * Hint: เรียก service.getDiscountedPrice(id)
108 * ทดสอบ: GET http://localhost:8080/products/1/price
109 */
110 @GetMapping("/{id}/price")
111 public Mono<Double> getDiscountedPrice(@PathVariable String id) {
112     return service.getDiscountedPrice(id);
113 }
114 }
115

```

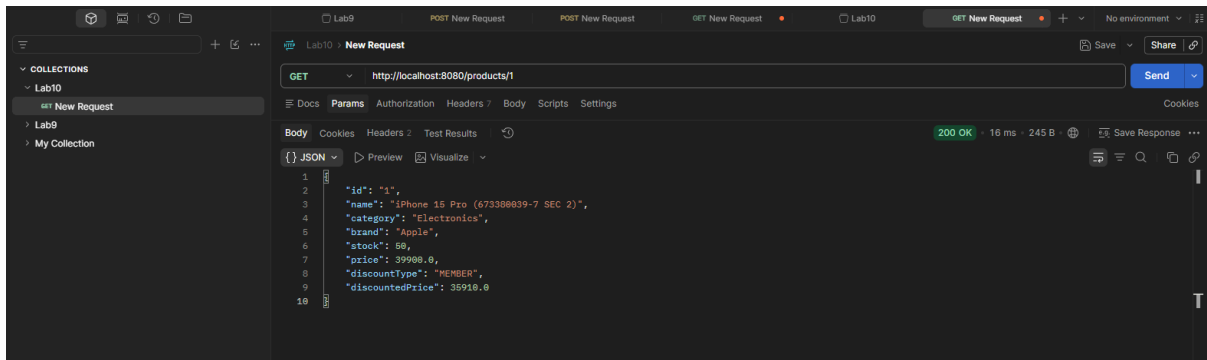
ProductController ทำหน้าที่รับ HTTP request แล้วเรียก method ใน Service ตาม endpoint ได้แก่ GET รายการทั้งหมด, GET ตาม id, POST เพื่อสร้างสินค้า, DELETE เพื่อลบสินค้า, GET ตาม category และ GET ราคาหลังส่วนลด ทุก method คืน Mono หรือ Flux โดยตรง จึงไม่มีการ block และไม่ใส่ business logic ไว้ใน Controller

ผลการทดสอบทุก Endpoint

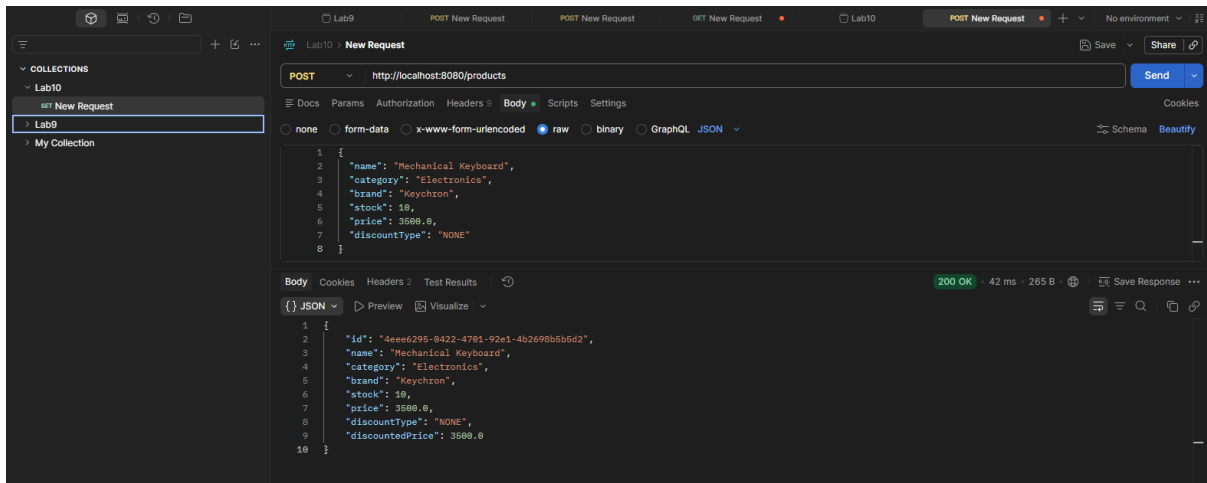
GET /products



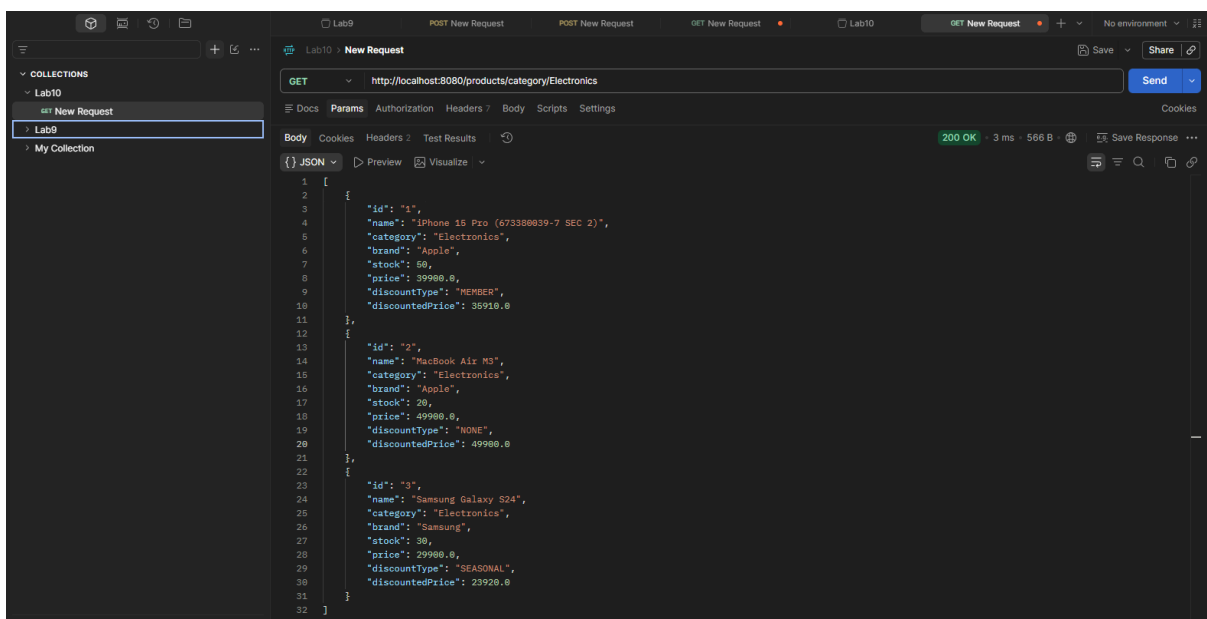
GET /products/1



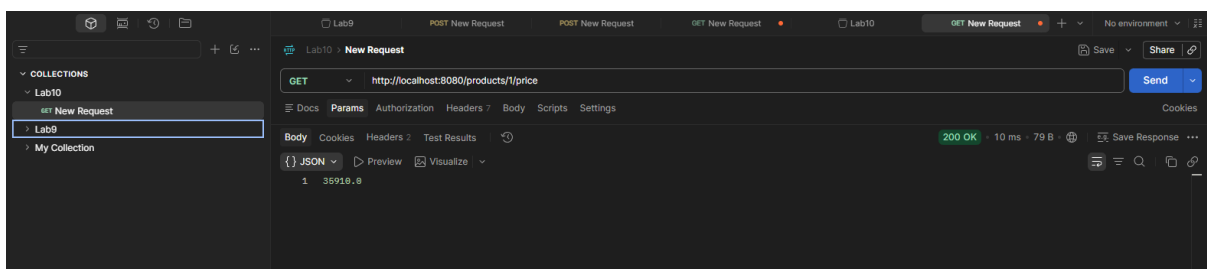
POST /products



GET /products/category/Electronics



GET /products/1/price



DELETE /products/{id}

